

Д311: Оркестрация пайплайнов с Argo Workflows

1. Шаблон: HTTP Request (Загрузка данных)

Шаблон для выполнения HTTP-запросов и сохранения результата. Часто используется для выгрузки данных из внешних API или скачивания файлов.

Входные параметры:

- `url` — адрес для запроса;
- `method` — HTTP-метод (по умолчанию `GET`).

Выходные данные:

- `response` (artifact) — файл с ответом сервера.

YAML-код:

```
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: http-request
spec:
  templates:
    - name: http-request
      inputs:
        parameters:
          - name: url
          - name: method
            value: "GET"
      outputs:
        artifacts:
          - name: response
            path: /tmp/response.txt
      container:
        image: curlimages/curl:latest
        command: [sh, -c]
        args: ["curl -X {{inputs.parameters.method}} -sL
{{inputs.parameters.url}} -o /tmp/response.txt"]
```

2. Шаблон: Python Transform (Обработка данных)

Универсальный шаблон для запуска Python-скриптов. Позволяет передать входной артефакт, выполнить над ним произвольный код и вернуть результат.

Входные параметры:

- `script` (parameter) — исходный код на Python;

- `input-data` (artifact) — входной файл для обработки.

Выходные данные:

- `output-data` (artifact) — результат работы скрипта.

YAML-код:

```
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: python-transform
spec:
  templates:
    - name: python-transform
      inputs:
        parameters:
          - name: script
        artifacts:
          - name: input-data
            path: /tmp/input.txt
      outputs:
        artifacts:
          - name: output-data
            path: /tmp/output.txt
      script:
        image: python:3.9-slim
        command: [python]
        source: "{{inputs.parameters.script}}"
```

3. Шаблон: Send Notification (Уведомление)

Шаблон для отправки уведомлений об успешном или ошибочном завершении пайплайна. В данном примере реализован через `echo` для простоты, но легко расширяется до Slack/Telegram API.

Входные параметры:

- `message` — текст уведомления.

Выходные данные:

- Нет.

YAML-код:

```
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: send-notification
```

```
spec:
  templates:
    - name: send-notification
      inputs:
        parameters:
          - name: message
      container:
        image: alpine:latest
        command: [sh, -c]
        args: ["echo 'Notification sent: {{inputs.parameters.message}}'"]
```

Основной Workflow (Пайплайн)

Основной **Workflow** объединяет созданные шаблоны в осмысленный ETL-процесс:

1. Скачивает данные по URL.
2. Обрабатывает их с помощью Python-скрипта (считает количество символов).
3. Отправляет уведомление об успешном завершении.

YAML-код:

```
apiVersion: argoproj.io/v1alpha1
kind: Workflow
metadata:
  generateName: data-pipeline-
spec:
  entrypoint: main
  templates:
    - name: main
      steps:
        - - name: fetch-data
            templateRef:
              name: http-request
              template: http-request
            arguments:
              parameters:
                - name: url
                  value: "https://raw.githubusercontent.com/argoproj/argo-
workflows/master/README.md"

        - - name: process-data
            templateRef:
              name: python-transform
              template: python-transform
            arguments:
              parameters:
                - name: script
                  value: |
import sys
with open('/tmp/input.txt', 'r') as f:
```

```
        data = f.read()
        with open('/tmp/output.txt', 'w') as f:
            f.write(f"Processed {len(data)} characters.\n")
artifacts:
  - name: input-data
    from: "{{steps.fetch-data.outputs.artifacts.response}}"

  - name: notify-success
    templateRef:
      name: send-notification
      template: send-notification
    arguments:
      parameters:
        - name: message
          value: "Pipeline finished successfully!"
```

Итог

Использование `WorkflowTemplate` позволяет вынести типовые задачи (загрузка, обработка, уведомления) в переиспользуемые блоки. Это делает основные `Workflow` более читаемыми, упрощает поддержку и позволяет собирать новые пайплайны как из конструктора.